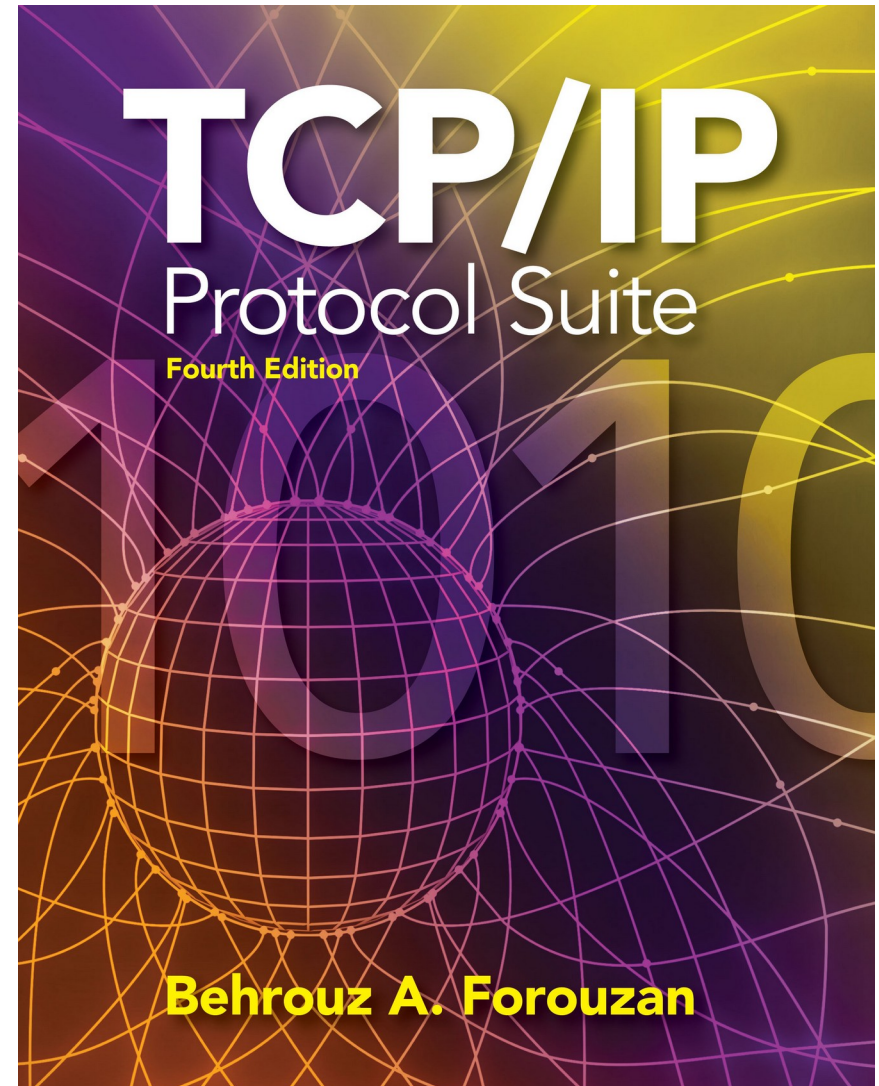


Chapter 14

User Datagram Program (UDP)



OBJECTIVES:

- ❑ **To introduce UDP and show its relationship to other protocols in the TCP/IP protocol suite.**
- ❑ **To explain the format of a UDP packet and discuss the use of each field in the header.**
- ❑ **To discuss the services provided by the UDP such as process-to-process delivery, multiplexing/demultiplexing, and queuing.**
- ❑ **To show how to calculate the optional checksum and the sender the needs to add a pseudoheader to the packet when calculating the checksum.**
- ❑ **To discuss how some application programs can benefit from the simplicity of UDP.**

Chapter Outline

14.1 *Introduction*

14.2 *User Datagram*

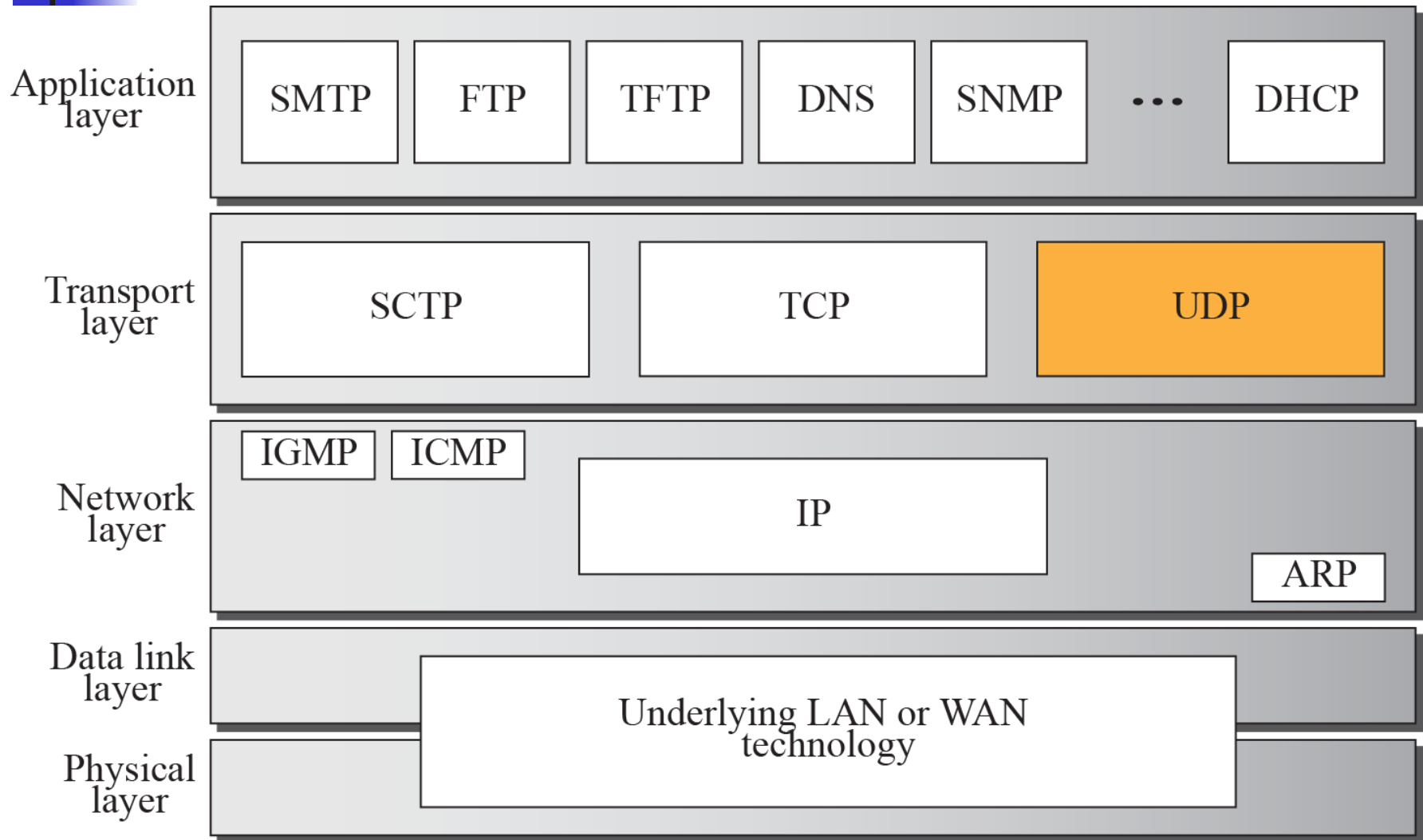
14.3 *UDP Services*

14.4 *UDP Application*

14-1 INTRODUCTION

Figure 14.1 shows the relationship of the User Datagram Protocol (UDP) to the other protocols and layers of the TCP/IP protocol suite: UDP is located between the application layer and the IP layer, and serves as the intermediary between the application programs and the network operations.

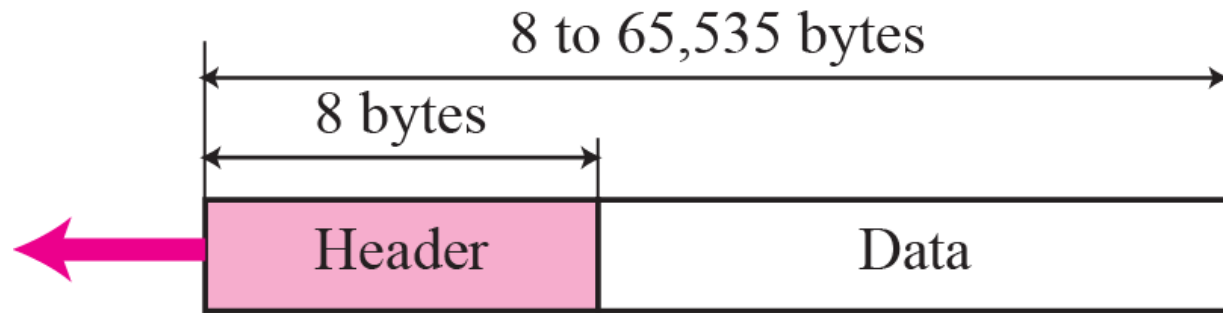
Figure 14.1 *Position of UDP in the TCP/IP protocol suite*



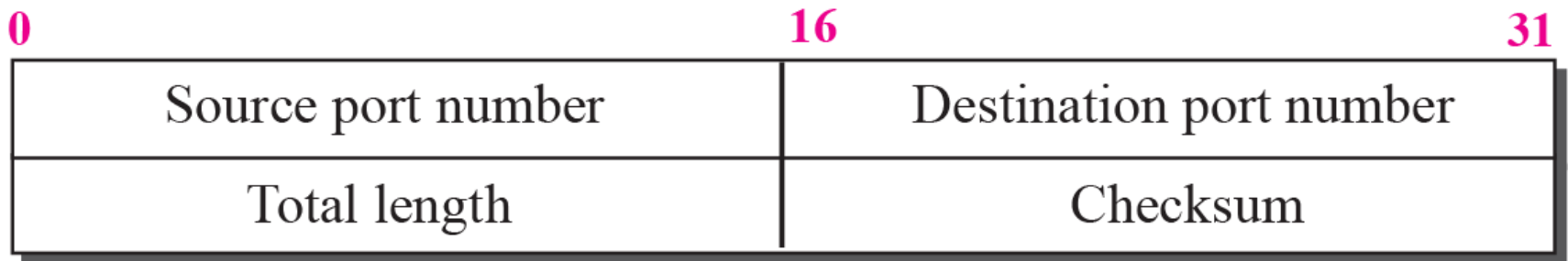
14-2 USER DATAGRAM

UDP packets, called user datagrams, have a fixed-size header of 8 bytes. Figure 14.2 shows the format of a user datagram.

Figure 14.2 *User datagram format*



a. UDP user datagram



b. Header format

Example 14.1

The following is a dump of a UDP header in hexadecimal format

```
CB84000D001C001C
```

- a.** What is the source port number?
- b.** What is the destination port number?
- c.** What is the total length of the user datagram?
- d.** What is the length of the data?
- e.** Is the packet directed from a client to a server or vice versa?
- f.** What is the client process?

Example 14.1 *Continued*

Solution

- a.** The source port number is the first four hexadecimal digits $(CB84)_{16}$ or 52100.
- b.** The destination port number is the second four hexadecimal digits $(000D)_{16}$ or 13.
- c.** The third four hexadecimal digits $(001C)_{16}$ define the length of the whole UDP packet as 28 bytes.
- d.** The length of the data is the length of the whole packet minus the length of the header, or $28 - 8 = 20$ bytes.
- e.** Since the destination port number is 13 (well-known port), the packet is from the client to the server.
- f.** The client process is the Daytime (see Table 14.1).

14-3 UDP Services

We discussed the general services provided by a transport layer protocol in Chapter 13. In this section, we discuss what portions of those general services are provided by UDP.

Topics Discussed in the Section

- ✓ **Process-to-Process Communication**
- ✓ **Connectionless Service**
- ✓ **Flow Control**
- ✓ **Error Control**
- ✓ **Congestion Control**
- ✓ **Encapsulation and Decapsulation**
- ✓ **Queuing**
- ✓ **Multiplexing and Demultiplexing**
- ✓ **Comparison between UDP and Generic Simple Protocol**

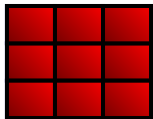
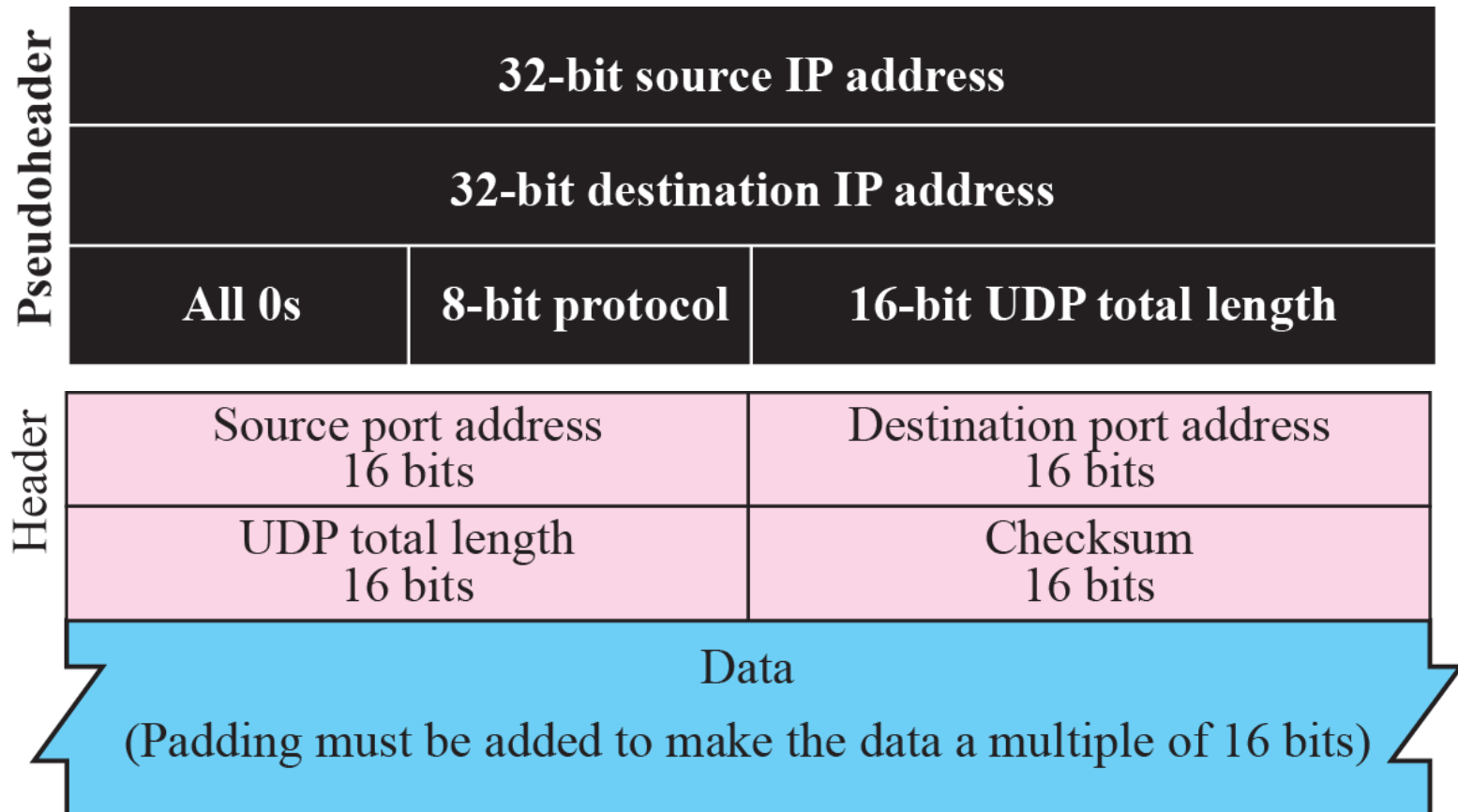


Table 14.1 *Well-known Ports used with UDP*

<i>Port</i>	<i>Protocol</i>	<i>Description</i>
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
53	Domain	Domain Name Service (DNS)
67	Bootps	Server port to download bootstrap information
68	Bootpc	Client port to download bootstrap information
69	TFTP	Trivial File Transfer Protocol
111	RPC	Remote Procedure Call
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management Protocol
162	SNMP	Simple Network Management Protocol (trap)

Figure 14.3 *Pseudoheader for checksum calculation*



Example 14.2

Figure 14.4 shows the checksum calculation for a very small user datagram with only 7 bytes of data. Because the number of bytes of data is odd, padding is added for checksum calculation. The pseudoheader as well as the padding will be dropped when the user datagram is delivered to IP (see Appendix F).

Figure 14.4 *Checksum calculation for a simple UDP user datagram*

153.18.8.105			
171.2.14.10			
All 0s	17	15	
1087		13	
15		All 0s	
T	E	S	T
I	N	G	Pad

10011001	00010010	→	153.18
00001000	01101001	→	8.105
10101011	00000010	→	171.2
00001110	00001010	→	14.10
00000000	00010001	→	0 and 17
00000000	00001111	→	15
00000100	00111111	→	1087
00000000	00001101	→	13
00000000	00001111	→	15
00000000	00000000	→	0 (checksum)
01010100	01000101	→	T and E
01010011	01010100	→	S and T
01001001	01001110	→	I and N
01000111	00000000	→	G and 0 (padding)
10010110	11101011	→	Sum
01101001	00010100	→	Checksum

Example 14.3

What value is sent for the checksum in one of the following hypothetical situations?

- a.** The sender decides not to include the checksum.
- b.** The sender decides to include the checksum, but the value of the sum is all 1s.
- c.** The sender decides to include the checksum, but the value of the sum is all 0s.

Example 14.3 *Continued*

Solution

- a.** The value sent for the checksum field is all 0s to show that the checksum is not calculated.
- b.** When the sender complements the sum, the result is all 0s; the sender complements the result again before sending. The value sent for the checksum is all 1s. The second complement operation is needed to avoid confusion with the case in part a.
- c.** This situation never happens because it implies that the value of every term included in the calculation of the sum is all 0s, which is impossible; some fields in the pseudoheader have nonzero values (see Appendix D).

Figure 14.5 *Encapsulation and decapsulation*

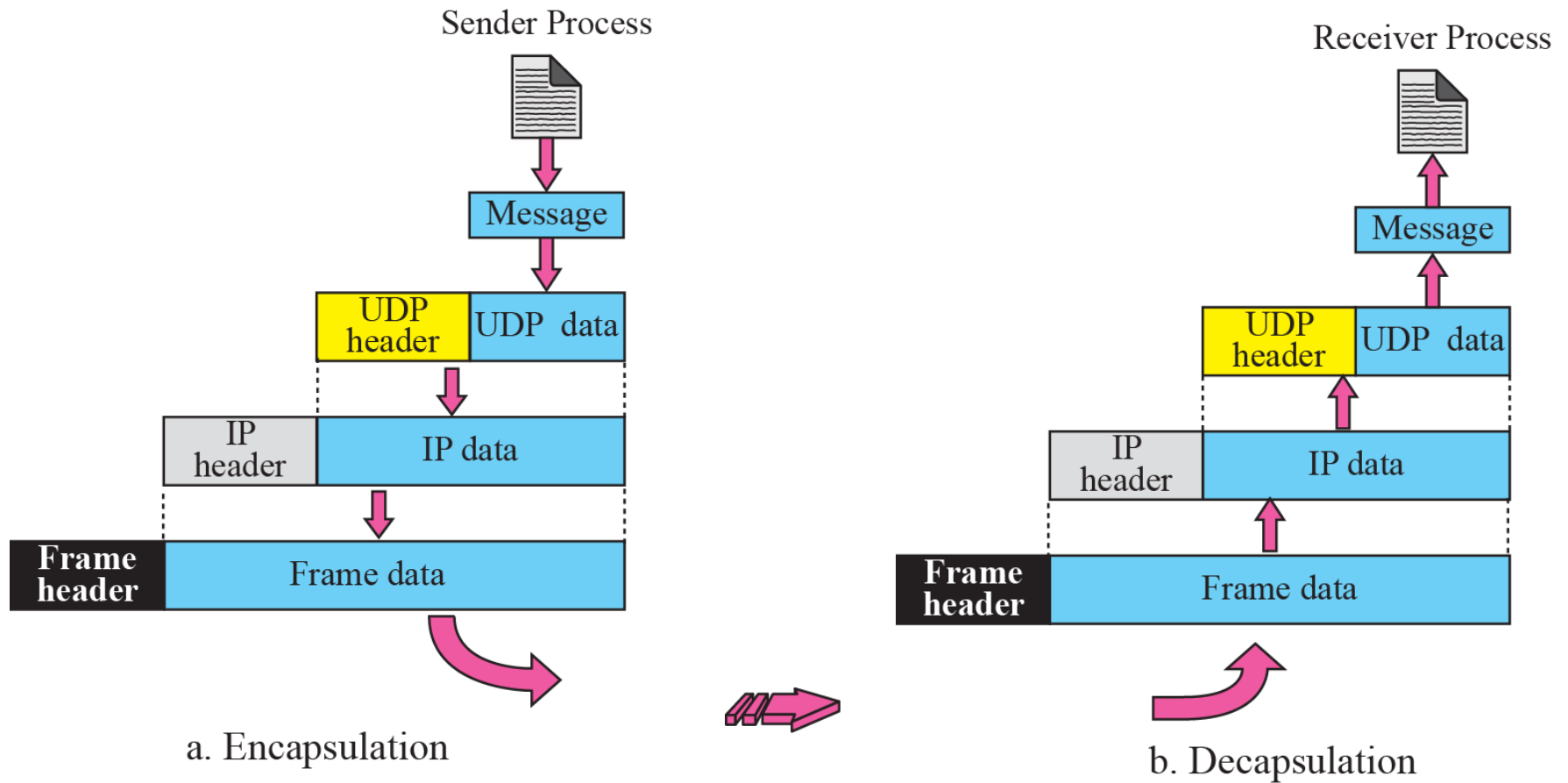


Figure 14.6 *Queues in UDP*

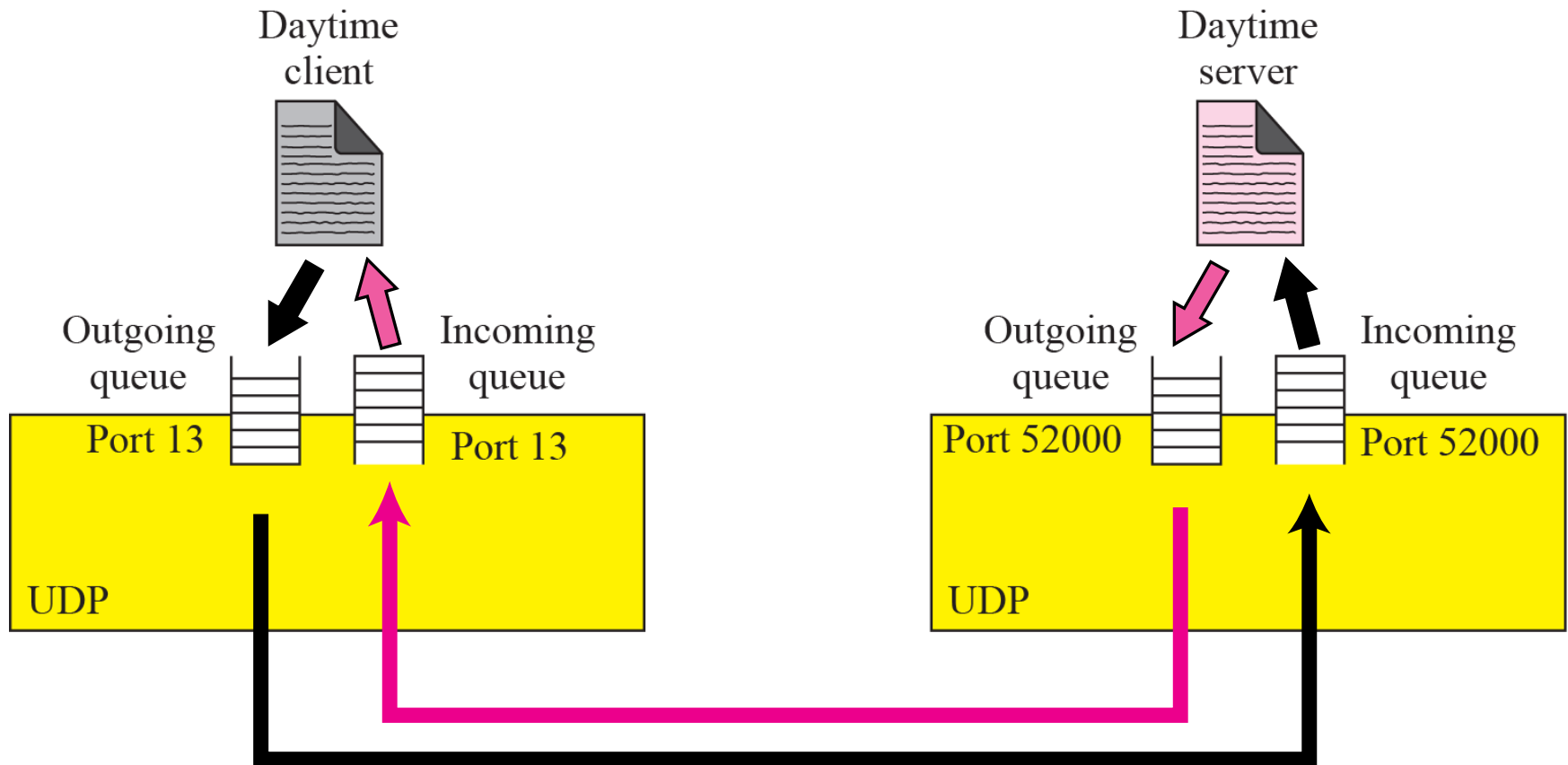
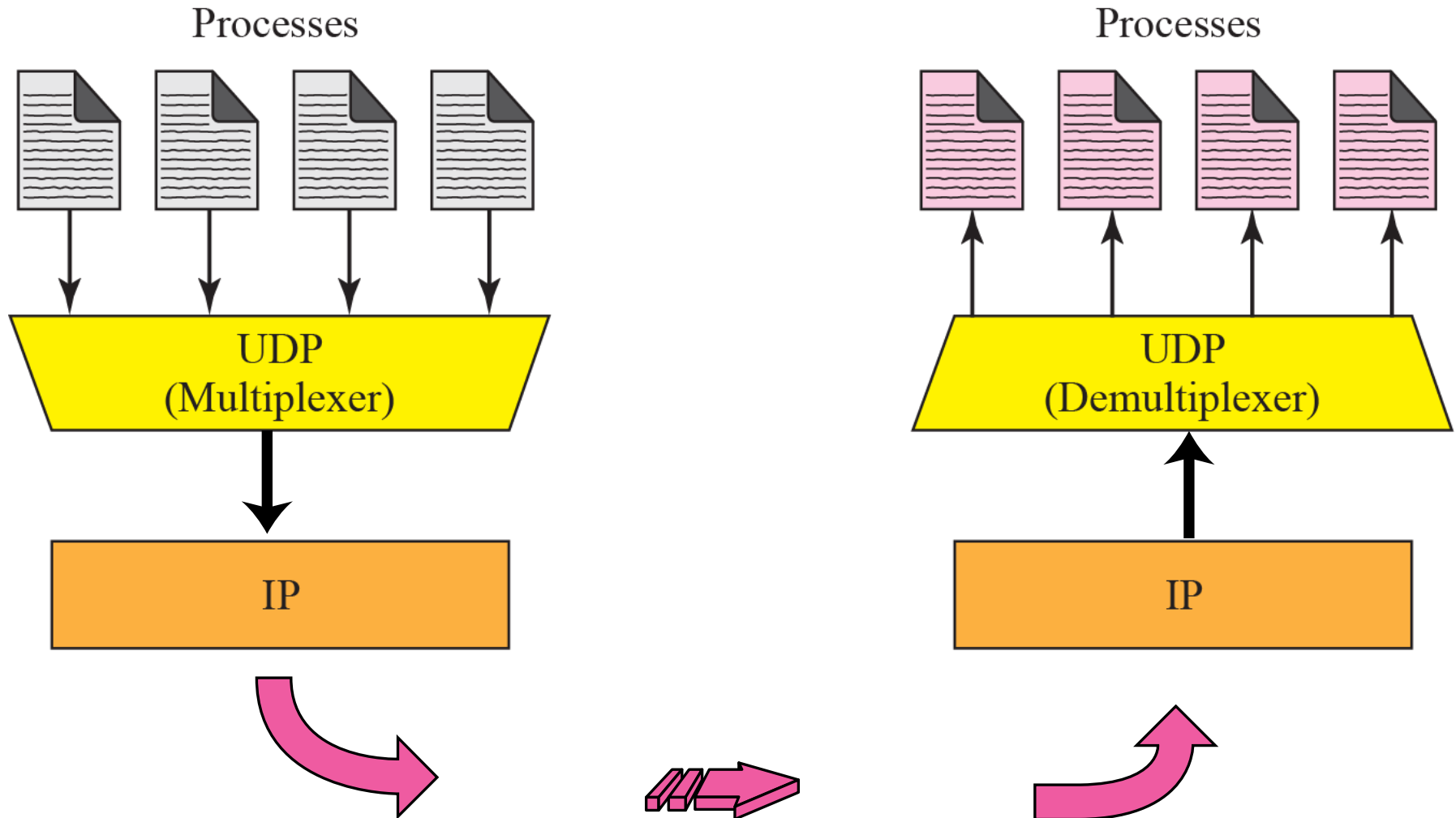


Figure 14.7 *Multiplexing and demultiplexing*



Note

UDP is an example of the connectionless simple protocol we discussed in Chapter 13 with the exception of an optional checksum added to packets for error detection.

14-4 UDP APPLICATION

Although UDP meets almost none of the criteria we mentioned in Chapter 13 for a reliable transport-layer protocol, UDP is preferable for some applications. The reason is that some services may have some side effects that are either unacceptable or not preferable. An application designer needs sometimes to compromise to get the optimum.

Topics Discussed in the Section

- ✓ **UDP Features**
- ✓ **Typical Applications**

Example 14.4

A client-server application such as DNS (see Chapter 19) uses the services of UDP because a client needs to send a short request to a server and to receive a quick response from it. The request and response can each fit in one user datagram. Since only one message is exchanged in each direction, the connectionless feature is not an issue; the client or server does not worry that messages are delivered out of order.

Example 14.5

A client-server application such as SMTP (see Chapter 23), which is used in electronic mail, cannot use the services of UDP because a user can send a long e-mail message, which may include multimedia (images, audio, or video). If the application uses UDP and the message does not fit in one single user datagram, the message must be split by the application into different user datagrams. Here the connectionless service may create problems. The user datagrams may arrive and be delivered to the receiver application out of order. The receiver application may not be able to reorder the pieces. This means the connectionless service has a disadvantage for an application program that sends long messages.

Example 14.6

Assume we are downloading a very large text file from the Internet. We definitely need to use a transport layer that provides reliable service. We don't want part of the file to be missing or corrupted when we open the file. The delay created between the delivery of the parts are not an overriding concern for us; we wait until the whole file is composed before looking at it. In this case, UDP is not a suitable transport layer.

Example 14.7

Assume we are watching a real-time stream video on our computer. Such a program is considered a long file; it is divided into many small parts and broadcast in real time. The parts of the message are sent one after another. If the transport layer is supposed to resend a corrupted or lost frame, the synchronizing of the whole transmission may be lost. The viewer suddenly sees a blank screen and needs to wait until the second transmission arrives. This is not tolerable. However, if each small part of the screen is sent using one single user datagram, the receiving UDP can easily ignore the corrupted or lost packet and deliver the rest to the application program. That part of the screen is blank for a very short period of the time, which most viewers do not even notice. However, video cannot be viewed out of order, so streaming audio, video and voice applications that run over